STAT395: Sampling

Richard Arnold et al.

2026-04-01

Contents


```r
library(haven)
library(dplyr)
```

```
##
## Attaching package: 'dplyr'

## The following objects are masked from 'package:stats':
##
##     filter, lag

## The following objects are masked from 'package:base':
##
##     intersect, setdiff, setequal, union
```

```r
library(ggplot2)
library(kableExtra)
```

```
##
## Attaching package: 'kableExtra'

## The following object is masked from 'package:dplyr':
##
##     group_rows
```

```r
library(survey)
```

```
## Loading required package: grid

## Loading required package: Matrix

## Loading required package: survival

##
## Attaching package: 'survey'

## The following object is masked from 'package:graphics':
##
##     dotchart
```

```r
library(srvyr)
```

```
##
## Attaching package: 'srvyr'

## The following object is masked from 'package:kableExtra':
##
##     group_rows

## The following object is masked from 'package:stats':
##
##     filter
```

```r
library(sampling)
```

```
##
## Attaching package: 'sampling'

## The following objects are masked from 'package:survival':
##
##     cluster, strata
```

```r
#library(gtsummary) # error installing/loading this
library(mice)
```

```
##
## Attaching package: 'mice'

## The following object is masked from 'package:stats':
##
##     filter

## The following objects are masked from 'package:base':
##
##     cbind, rbind
```

```r
#library(simputation) # error installing/loading this
```

Chapter 1

Introductory Remarks

These are notes for the Sampling section of STAT395/455. The course overlaps to some extent with STAT392/439, but there is a greater emphasis on R coding, computation, summary and display.

1.1 Lecture Plan

Course Learning Objectives

- Describe the steps in the survey process, including the sources of survey error;
- Draw samples from a sample frame under a variety of sample designs, and compute survey weights;
- Carry out, interpret and present statistical inference procedures on survey data, including adjustments for non-response.

Lecture plan (3 lectures + 1 tutorial per week)

Bring a laptop to the Friday (tutorial) lecture.

Week 1 – The Survey Process

- The survey process, sources of survey error
- Populations and Frames
- Sampling in finite and infinite populations; Basics of SRSWOR inference; sample weights;
- **Tutorial:** Sampling basics; Population and Frames

Week 2 – Sampling

Use of `sample()` and other simulation methods in R for a variety of designs [`sampler` package (does power calculations, allocation, SRS and STSRS, but not cluster)] [`sampling` package – can do cluster samples]

- Simple random sampling, with and without replacement
- Stratified sampling
- Cluster sampling
- Sampling in complex designs
- Sample size calculation and allocation
- Survey weights – create estimates from weights without standard errors

- **Tutorial:** using R to draw samples and compute survey weights

Week 3 - Inference with the R survey package

- Inference in simple and complex designs, including regression
- Variance estimation
- Definition and interpretation of the design effect
- **Tutorial:** using R to specify designs and draw inferences

Week 4 – Non-response and complex designs

- Post-stratification for non-response adjustment
- Imputation (`simputation` package)
- Jackknife and bootstrap inference
- **Tutorial:** raking adjustments to weights; imputation; jackknife inference

Assignment 4 (10%) [due end of Week 12: covers material from Weeks 9-11]

- Survey process, populations and frames
- Describe a real survey, comment on design
- Propose a sample design, frame, comment on scope and coverage
- Draw and analyse a sample under various designs
- Compute sample sizes and allocate samples in stratified sampling
- Compute and interpret the design effect
- Analyse data and present results from a survey

Test 2 (30%) [assessment period] [1.5 hours, closed book]

- Survey process, populations and frames
- Describe sample designs
- Compute and interpret sample weights
- Use sample weights for point estimates
- Interpret output from survey inference in R
- Compare results, interpret design effect
- Describe methods for non-response adjustment (imputation and post-stratification); comment on bias reduction and assumptions being made
- Describe the jackknife method of variance estimation

1.2 Recommended reading

There are lots of books on sample surveys.

- Thomas Lumley's book 'Complex Surveys' (Lumley (2010)) is a good introduction to sample survey analysis with R, and accompanies the `survey` package (Lumley (2004)).
- Groves et al. 'Survey Methodology' (groves.etal) is a good general summary of the important issues in survey design.
- Sarndal, Swensson and Wretman is a classic in the field of the theory of sampling (Särndal et al. (1992))
- Likewise the books by Cochran (1977) and Kish (1965)
- Little and Rubin have a great book on missing data (Little and Rubin (2002))
- More recent are the books by Lohr (1999) and Scheaffer et al. (2006)
- And in 1994 StatsNZ published 'A Guide to Good Survey Design' (Zealand (1995))

And also

- CRAN site on R resources for Official Statistics https://cran.r-project.org/web/views/OfficialStatistics.html
- Online book on using R for survey analysis: https://bookdown.org/jimr1603/Intermediate_R_-_R_for_Survey_Analysis/

Some simple lectures on Sample Surveys

- https://r-project.ro/conference2017/presentations/Camelia_Goga_sondages.pdf
- Notes for epidemiologists: https://www.epirhandbook.com/en/new_pages/survey_analysis.html
- Notes on imputation: https://www.r-bloggers.com/2023/01/imputation-in-r-top-3-ways-for-imputing-missing-data/

Data

- UC Irvine Machine Learning data repository: http://archive.ics.uci.edu/
- Some free microdata sets: https://asdfree.com/
- Kaggle data repository: https://www.kaggle.com/

1.3 R Packages

- `survey` package (Lumley (2004)) with this handy summary of command uses https://stats.oarc.ucla.edu/r/seminars/survey-data-analysis-with-r/
- `srvyr` a dplyr wrapper for the `survey` package
- `gtsummary` creates outputs ready for publication (difficult to build/load)
- `sampling` package for drawing samples according to various designs
- `mice` package for imputation
- `simputation` package for imputation (difficult to build/load)
- `haven` package to import `SAS`, `Stata` and `SPSS` data files

Chapter 2

Sampling

In statistics the data we analyse is often a subset, i.e. a **sample** of members of a **population**.

> For example we might have a sample of households whose members respond to a questionnaire, or a sample of plants from a field.

In order to select from a population we first enumerate (list) the popualation in some way and then make a random selection of our subset members from that list. The method by which we do so is called the **sample design**.

Our focus in statistics is often the **estimation** of population characteristics (known as **population parameters**) on the basis of sample characteristics (which are called **sample statistics**),

When generating **simulated** datasets we also generate samples. Simulation is often done as a means of testing out statistical ideas and methods, or (in the case of the bootstrap method) when conventional ways of estimating standard errors (and therefore confidence intervals) don't work.

Before we get into detail of estimation and inference, let's spend some time thinking about how sampling works.

2.1 Random numbers

In both settings (sampling and simulation) we need to start with a method of generating random numbers. We'll start by assuming we have some means of generating a random sequence of numbers uniformly distributed in the interval [0,1]:

$$U_i \sim \text{Uniform}(0,1) \qquad \text{for } i = 1, 2, 3, ...$$

Here 'uniformly distributed' means that all values in the range [0,1] are equally likely to occur.

In R this is done using the `runif()` function. For example to generate 5 random draws from Uniform(0,1):

```
runif(5)
```

```
## [1] 0.2875775 0.7883051 0.4089769 0.8830174 0.9404673
```

Another draw will a different set of random numbers:

```
runif(5)
```

```
## [1] 0.0455565 0.5281055 0.8924190 0.5514350 0.4566147
```

If we draw a histogram of 100 draws it looks a bit lumpy:

```
hist(runif(100), xlab="x", ylab="Frequency")
```

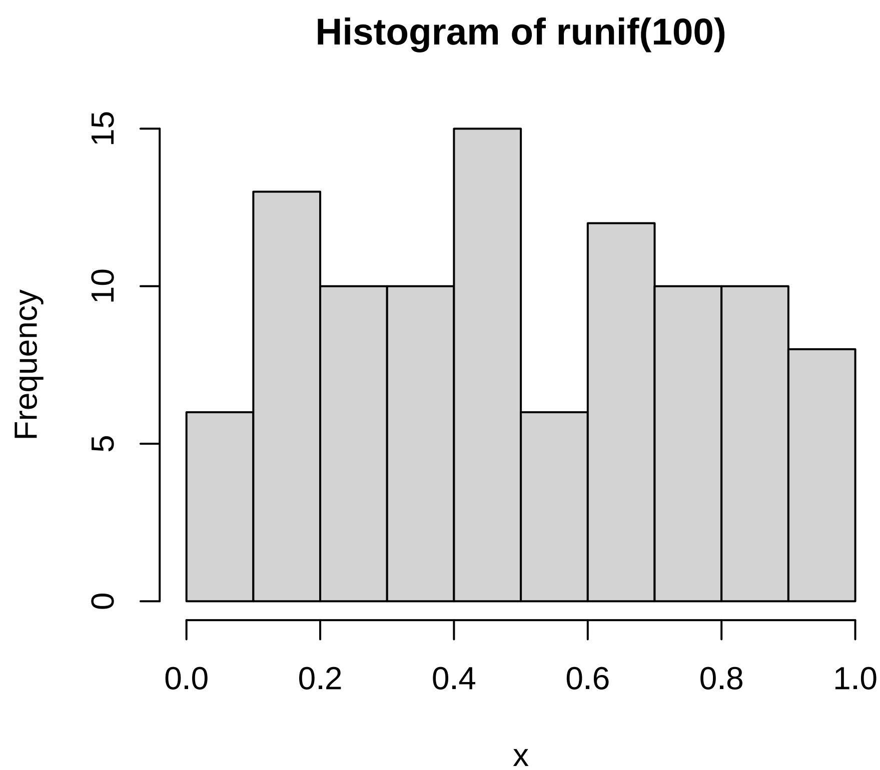

Figure 2.1: 100 draws from Uniform(0,1)

But a histogram of 100,000 draws is very uniform:

```
hist(runif(100000), xlab="x", ylab="Frequency")
```

Although the numbers appear to be random they are in fact generated using formulae, so if one knows the formula the sequence of random numbers generated by `runif()` is completed predictable. However the formula is complex enough that there are no easily discernible patterns in sequences of many millions of draws. These numbers are **psuedo-random**, but they are random enough for our purposes.

Underlying the random number generator is a **seed** value, which initiates the sequence. If we set the seed to a particular value, the sequence of random numbers that will be generated after doing so will always be the same.

```
set.seed(111)
runif(5)
```

```
## [1] 0.5929813 0.7264811 0.3704220 0.5149238 0.3776632
```

```
runif(5)
```

```
## [1] 0.41833733 0.01065785 0.53229524 0.43216062 0.09368152
```

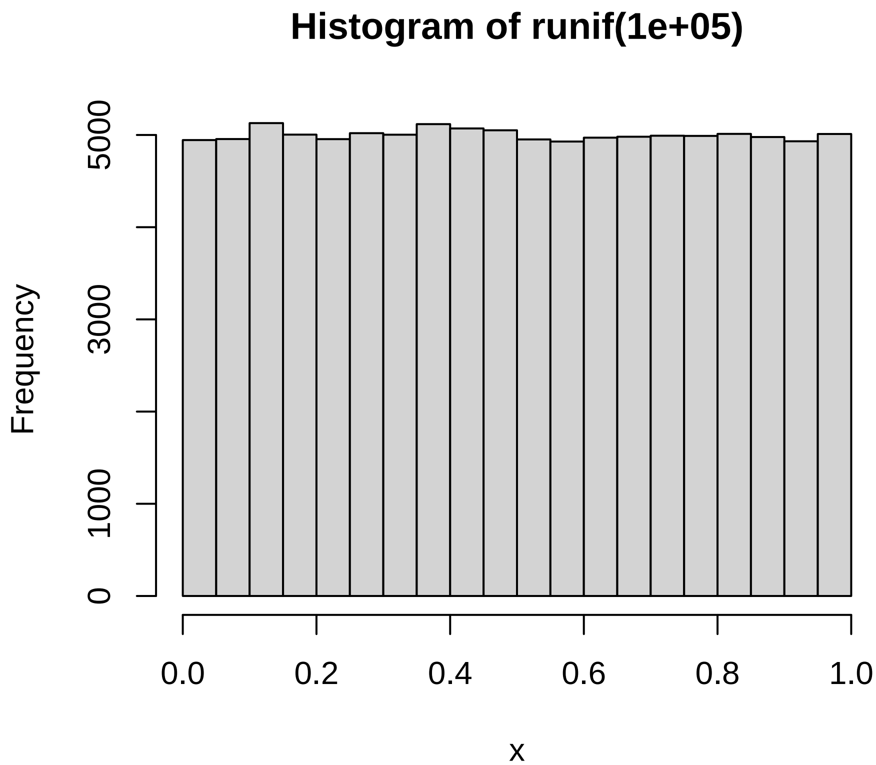

Figure 2.2: 100000 draws from Uniform(0,1)

```
runif(5)
```

```
## [1] 0.55577991 0.59022849 0.06714114 0.04754785 0.15620252
```

```
set.seed(8)
runif(5)
```

```
## [1] 0.4662952 0.2078233 0.7996580 0.6518713 0.3215092
```

```
set.seed(111)
runif(5)
```

```
## [1] 0.5929813 0.7264811 0.3704220 0.5149238 0.3776632
```

This property can be very useful when testing and comparing algorithms: we can generate an identical sequence of random numbers and check (for example) that two methods of estimation give the same results on two identical randomly generated datasets.

2.2 Sampling from categorical random variables

Draws from a uniform distribution are all we need to simulate/sample from any distribution.

Heads or tails: if the probability of flipping heads is 0.5, we can generate `n` random numbers, and then label any values less than 0.5 as Heads and the remainder as tails.

```
n <- 10
u <- runif(n)
u
```

```
##  [1] 0.2875775 0.7883051 0.4089769 0.8830174 0.9404673 0.0455565 0.5281055
##  [8] 0.8924190 0.5514350 0.4566147
```

```
y <- ifelse(u<=0.5,"Heads","Tails")
y
```

```
##  [1] "Heads" "Tails" "Heads" "Tails" "Tails" "Heads" "Tails" "Tails" "Tails"
## [10] "Heads"
```

```
table(y)
```

```
## y
## Heads Tails
##     4     6
```

```
table(ifelse(runif(10000)<=0.5,"Heads","Tails"))
```

```
##
## Heads Tails
##  5059  4941
```

We can extend this idea to any set of categories.

Red, Green or Blue: Say we have a six sided die: three sides are red, two are green and one is blue. The probabilities of rolling the three colours are therefore $\frac{1}{2}$, $\frac{1}{3}$, $\frac{1}{6}$ respectively. These probabilities add to one.

The random variable Y is the colour that the die shows when we roll it.

```
pp <- c(Red=1/2, Green=1/3, Blue=1/6)
sum(pp)
```

```
## [1] 1
```

In general we might have m categories, where the probability of that Y category k is p_k: $\Pr(Y = k) = p_k$. and

$$\sum_{k=1}^{m} p_k = 1$$

We then compute the **cumulative probabilities** c_k, the partial sums of these probabilities:

$$c_k = \sum_{j=1}^{k} p_j$$

The final cumulative probability c_m is always 1 by definition.

Implicitly we also have a value $c_0 = 0$, which is the the empty sum and always equal to zero.

These cumulative probabilities c_k are 'the probability that random variable Y has a category label less than or equal to k:

$$\Pr(Y \leq k) = c_k = \sum_{j=1}^{k} p_j$$

Red, Green or Blue: the cumulative probabilities are

$$\begin{aligned}
c_0 &= 0.0000 \\
c_1 &= p_1 = 0.5000 \\
c_2 &= p_1 + p_2 = 0.5000 + 0.3333 = 0.8333 \\
c_3 &= p_1 + p_2 + p_3 = 0.5000 + 0.3333 + 0.1667 = 1.0000
\end{aligned}$$

```
m <- length(pp) # Number of categories
cp <- cumsum(pp)
cp
```

```
##       Red     Green      Blue
## 0.5000000 0.8333333 1.0000000
```

We then take a set of uniform random numbers $u_1, \ldots, u_n$. We then want to find which of the m labels corresponds to each random number u_i.

The rule is that if u_i is less than cumulative probability c_k but greater than c_{k-1}, then we assign the label k.

There are various ways to do this, here's a version using `case_when()` from the `dplyr` package:

```
set.seed(112)
u <- runif(8)
u
```

```
## [1] 0.37667253 0.92201944 0.99187774 0.97723602 0.23629728 0.10706678 0.03915213
## [8] 0.72802690
```

```
y <- case_when(u <= cp[1] ~ "Red",
               u <= cp[2] ~ "Green",
               .default="Blue")    # here .default means "otherwise"
data.frame(round(u,5),y)
```

```
##   round.u..5.     y
## 1     0.37667   Red
## 2     0.92202  Blue
## 3     0.99188  Blue
## 4     0.97724  Blue
## 5     0.23630   Red
## 6     0.10707   Red
## 7     0.03915   Red
## 8     0.72803 Green
```

```
table(y)
```

```
## y
##  Blue Green   Red
##     3     1     4
```

```
table(y)/length(y)
```

```
## y
##  Blue Green   Red
## 0.375 0.125 0.500
```

Note that the categories in the table haven't come out in the Red-Green-Blue order that we want - instead they're in the default alphabetical order.

We can fix this by converting `y` to a `factor` object:

```
y <- factor(y,levels=names(pp))
table(y)
```

```
## y
##   Red Green  Blue
```

```
##      4      1      3
```

Here's what is really going on: we if we plot the cumulative probabilities as a staircase – a **cumulative distribution function (CDF)** then we draw u_i from a Uniform(0,1) distribution, and then find where a horizontal line drawn at the level of u_i intersects the CDF: and that is the value of the categorical variable y_i.

The larger the probability p_k the bigger the jump in the staircase, and the larger the probability that category k will be selected.

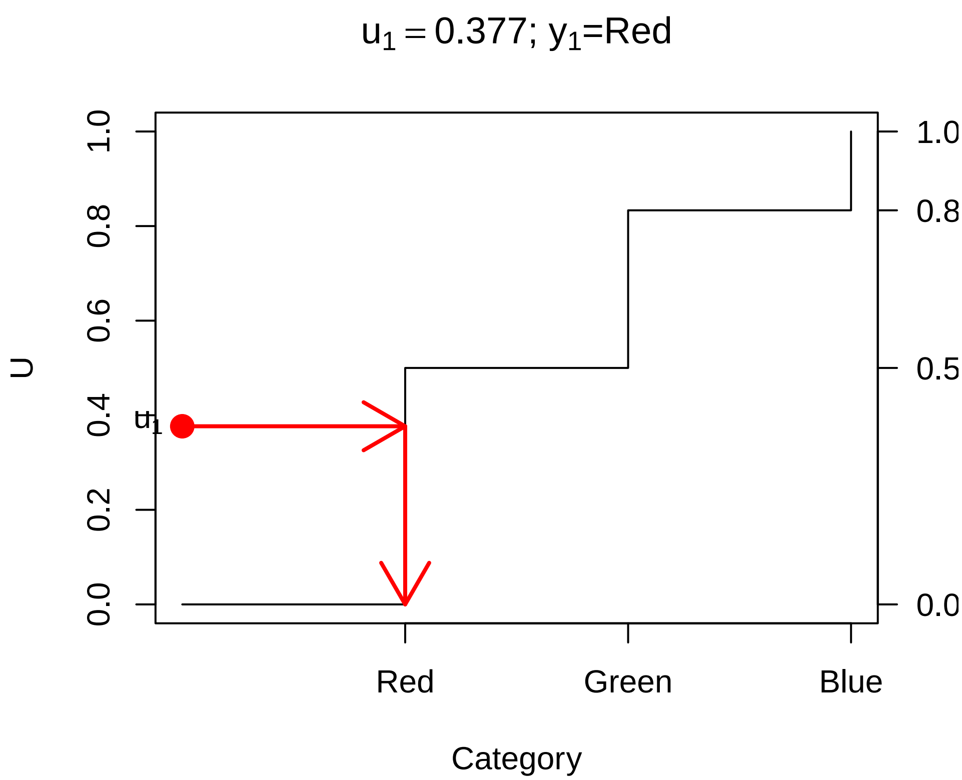

Figure 2.3: Drawing a random category

```
set.seed(112)
n <- 10000
u <- runif(n)
y <- case_when(u <= cp[1] ~ "Red",
               u <= cp[2] ~ "Green",
               .default="Blue")
y <- factor(y, levels=names(pp))
table(y)
```

```
## y
##   Red Green  Blue
##  4976  3336  1688
```

The `sample()` function implements this conveniently: select from the integers 1 to `m`

```
n <- 10000
y <- sample(m, size=n, prob=pp, replace=TRUE)
table(y)
```

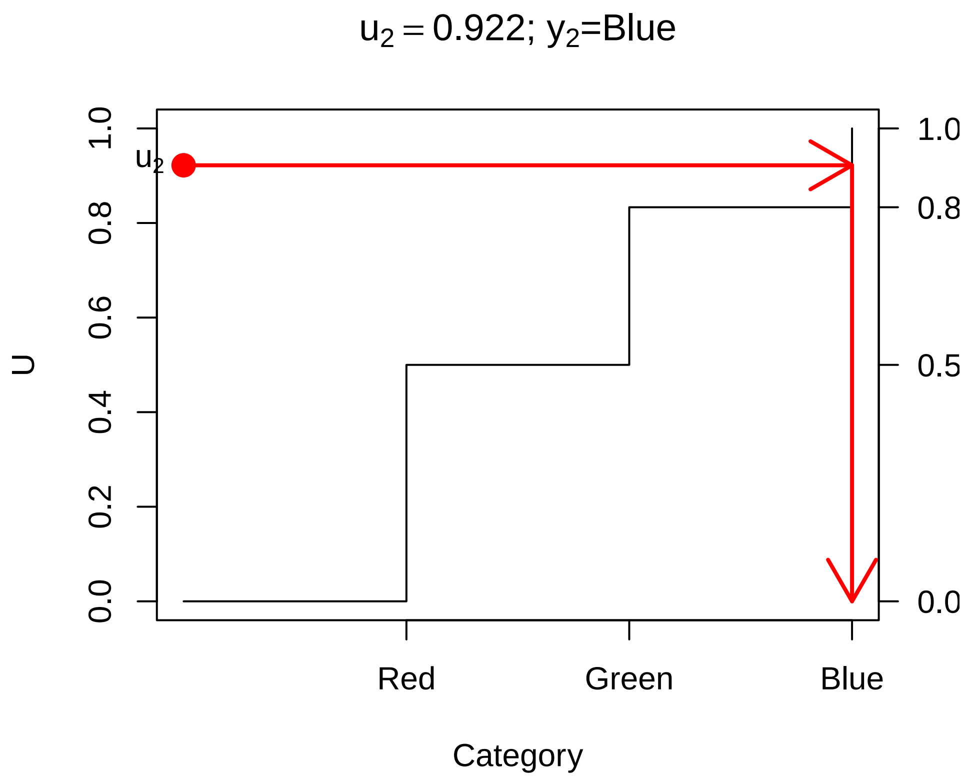

Figure 2.4: Drawing a random category

```
## y
##    1    2    3
## 5014 3350 1636
```

(need `replace=TRUE` here, see below).

... or state the vector `1:m` explicitly

```
y <- sample(1:m, size=n, prob=pp, replace=TRUE)
table(y)
```

```
## y
##    1    2    3
## 5010 3371 1619
```

... or directly from the vector of labels ...

```
y <- sample(c("Red","Green","Blue"), size=n, prob=pp, replace=TRUE)
y <- factor(y,levels=names(pp))
table(y)
```

```
## y
##   Red Green  Blue
##  4980  3325  1695
```

... get the labels from `pp` ...

```
y <- sample(names(pp), size=n, prob=pp, replace=TRUE)
y <- factor(y,levels=names(pp))
```

```
table(y)
```

```
## y
##   Red Green  Blue
##  4994  3389  1617
```

2.3 Sampling from discrete numerical random variables

The ideas above carry straight over to discrete numerical random variables.

The Binomial distribution $Y \sim \text{Bin}(m, p)$ is the number of successes in m trials where the probability of success on each trial is the same p, and all the trials are independent.

Y takes values 0 to m, and those probabilities can be calculated by

$$P(Y = k) = \binom{m}{k} p^k (1-p)^{m-k} = \frac{m!}{k!(m-k)!} p^k (1-p)^{m-k}$$

R provides these probabilities in the function `dbinom()`

e.g. $m = 10$ rolls of a six-sided die, and Y is the number of times we get a 1, so $p = 1/6$

```
m <- 10
p <- 1/6
pp <- dbinom(0:m, m, p)
pp
```

```
##  [1] 1.615056e-01 3.230112e-01 2.907100e-01 1.550454e-01 5.426588e-02
##  [6] 1.302381e-02 2.170635e-03 2.480726e-04 1.860544e-05 8.269086e-07
## [11] 1.653817e-08
```

```
barplot(pp, names=0:m,
        xlab="y", ylab="Pr(Y=y)",
        main=bquote("Bin("*.(m)*","*.(sprintf(" %.4f",p))*")"))
```

The CDF is given by the function `pbinom()`

```
cp <- pbinom(0:m, m, p)
```

```
plot(0:m, cp, type="s",
        xlab="y", ylab="Pr(Y<=y)",
        main=bquote("Bin("*.(m)*","*.(sprintf(" %.4f",p))*")"))
```

Drawing a single value of Y proceeds the same way as for a categorical variable:

So we can draw multiple values of Y from $\text{Bin}(10, \frac{1}{6})$ using the `sample()` function:

```
n <- 10000
pp <- dbinom(0:m, m, p)
y <- sample(0:m, n, prob=pp, replace=TRUE)
barplot(table(y))
```

Of course R provides a specific function for doing the same thing: `rbinom()`

```
n <- 10000
y <- rbinom(n, m, p)
barplot(table(y))
```

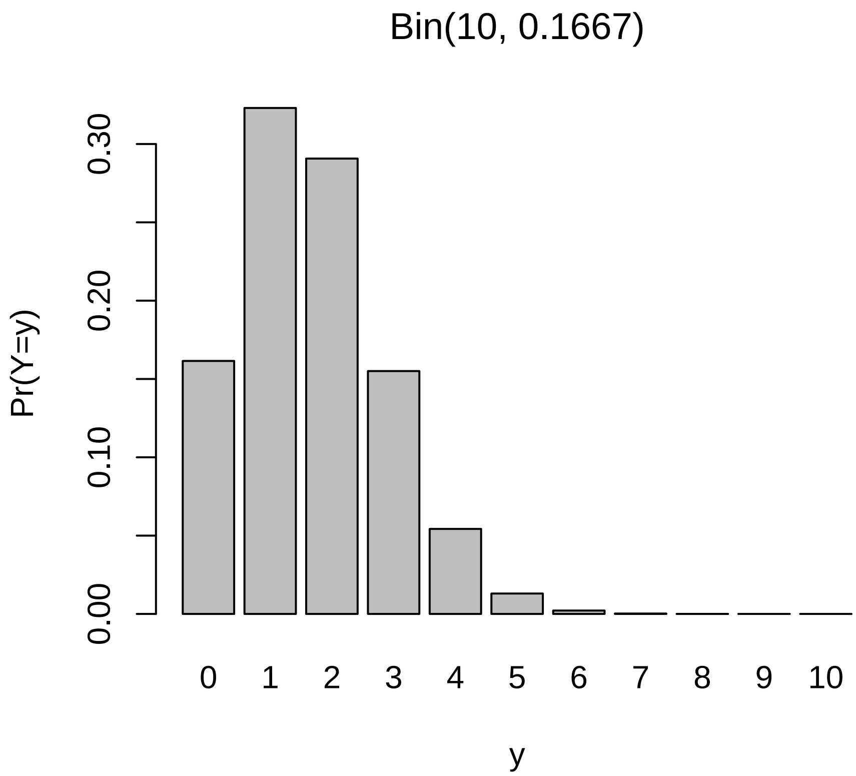

Figure 2.5: Binomial probabilities

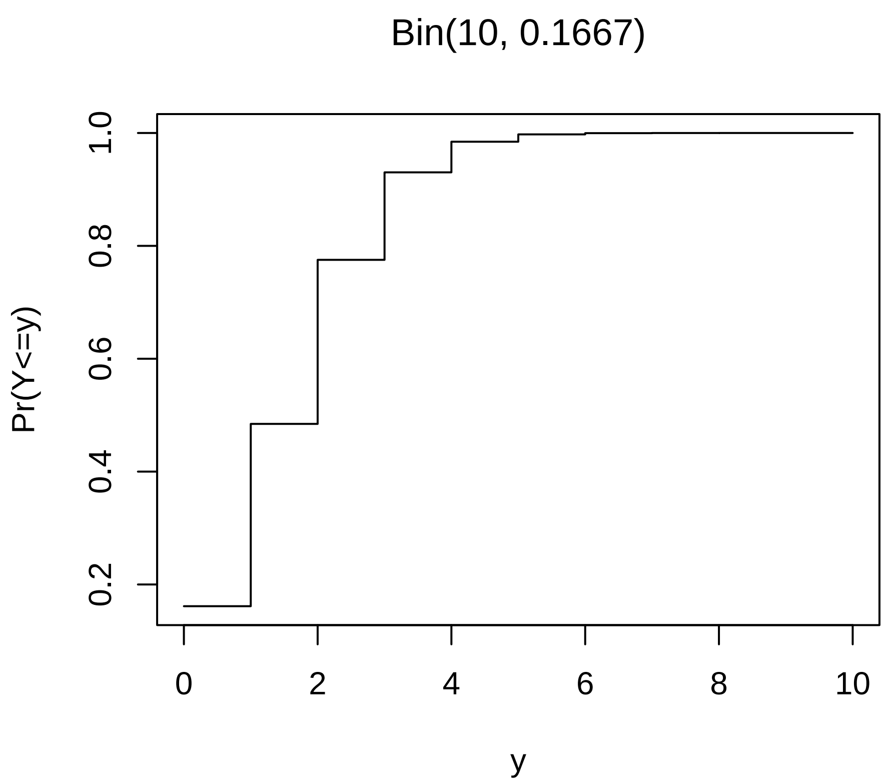

Figure 2.6: Binomial CDF

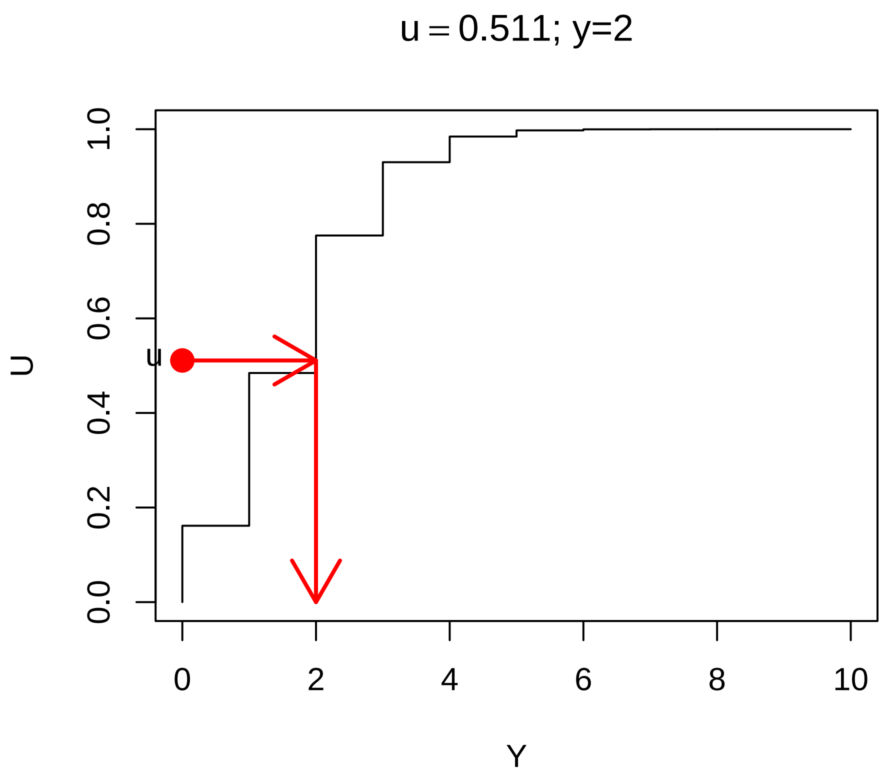

Figure 2.7: Drawing from a Binomial(10,1/6) distribution

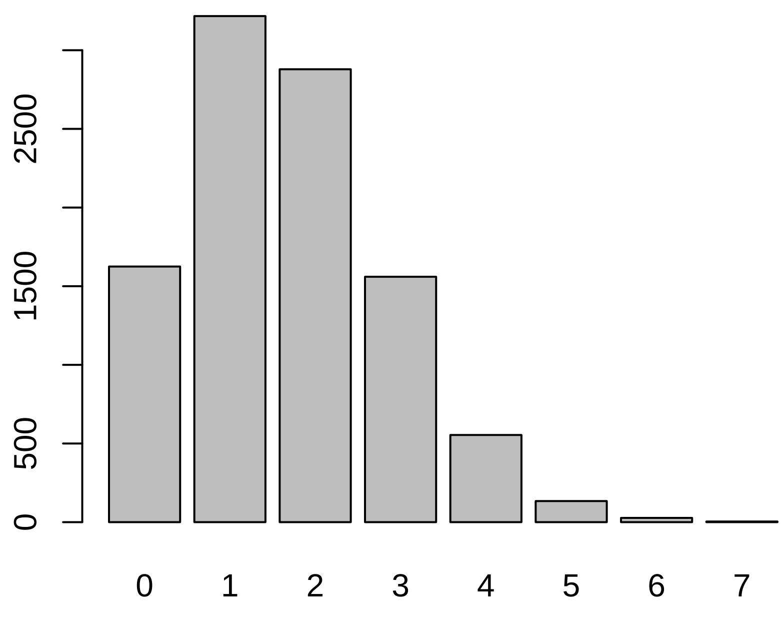

Figure 2.8: 10000 draws from a Binomial(10,1/6)

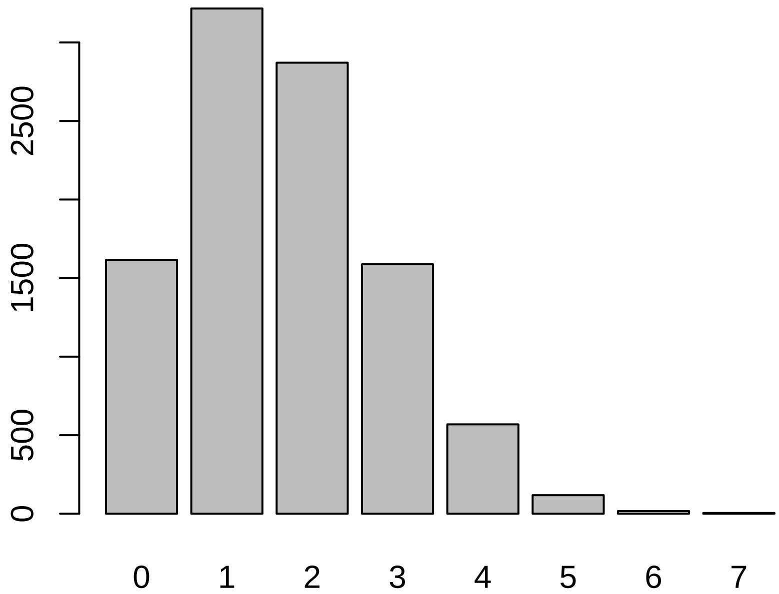

Figure 2.9: 10000 draws from a Binomial(10,1/6)

But that function is just doing what we've described above using `sample()`.

A particular common example is where we have m objects and want to select one of them at random, with each object having equal probability of being selected: i.e. each has probability $1/m$.

2.4 Sampling from continuous random variables

Continous random variables are simulated using the same principle.

The cumulative distribution function (CDF) of a continuous random variable Y is defined to be

$$F(y) = \Pr(Y \leq y) = \int_{-\infty}^{y} f(y')\, \mathrm{d}y'$$

We can use $F(y)$ to draw a random value from the distribution of Y by first drawing a random value U from a Uniform(0,1) distribution.

We then solve the equation

$$U = F(y)$$

for y: i.e. find the value of y corresponding to a CDF value of U.

For many standard distributions (including the Normal and Gamma distributions) there is no analytic formula which gives the solution for y given u: but there are numerical approximations which can be used in those settings.

One simple case which does have an analytic solution is the Exponential distribution $Y \sim \mathrm{Exp}(\lambda)$. This is a one parameter distribution where Y is non-negative ($Y \geq 0$), and can be used to model cases such the length of time it takes for an event to occur (e.g. the length of time for a radioactive nucleus to decay). The single parameter λ is the **rate** (e.g. number of radioactive decay events per unit time).

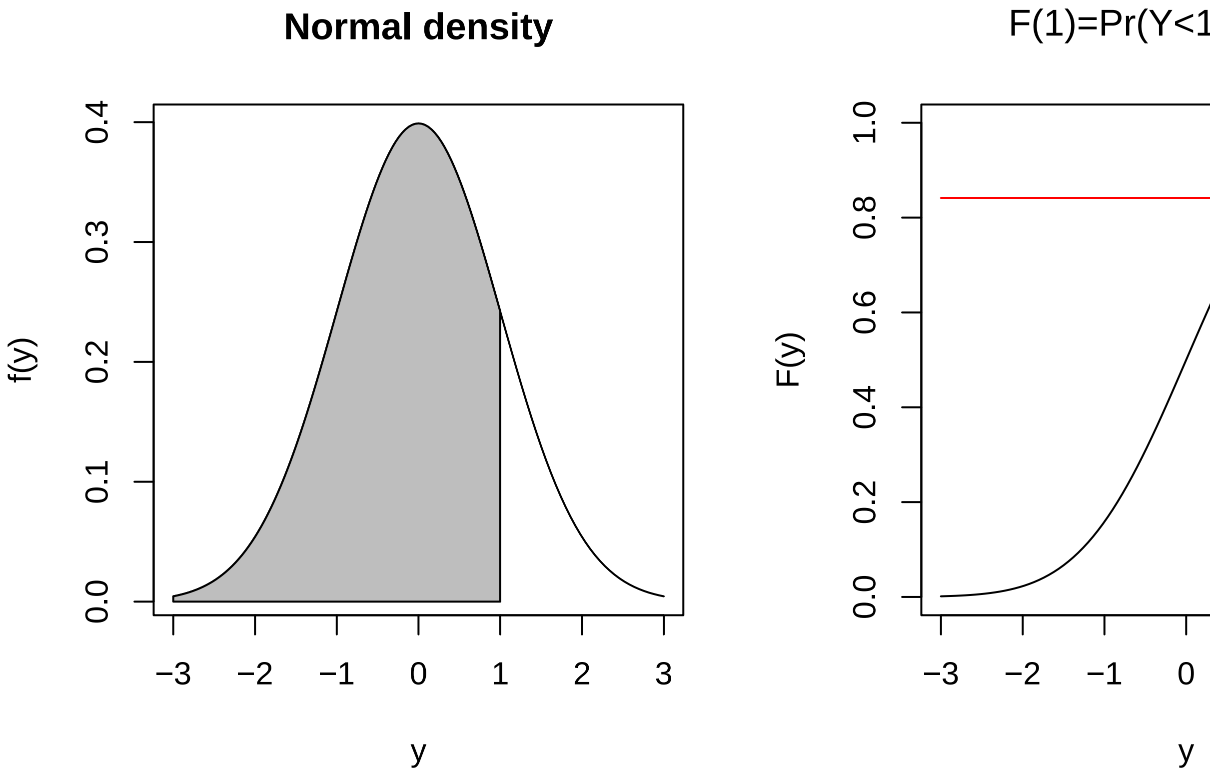

Figure 2.10: Normal distribution density and CDF

The exponential distribution has probability density function

$$f(y) = \lambda e^{-\lambda y} \qquad \text{where } y \geq 0$$

and has mean $E[Y] = 1/\lambda$.

The CDF is the integral of the probability density function

$$\Pr(Y \leq y) = F(y) = \int_0^y f(y)\,\mathrm{d}y$$

which for the Exponential distribution is

$$F(y) = \int_0^y \lambda e^{-\lambda t} \mathrm{d}t = 1 - e^{-\lambda y}$$

Solving

$$u = F(y) = 1 - e^{-\lambda y}$$

for y leads to

$$y = -\frac{1}{\lambda} \log(1 - u)$$

Using this function we can easily draw directly from an Exponential distribution.

```
n <- 10000
u <- runif(n)
lambda <- 0.5
y <- -(1/lambda)*log(1-u)
mean(y)
```

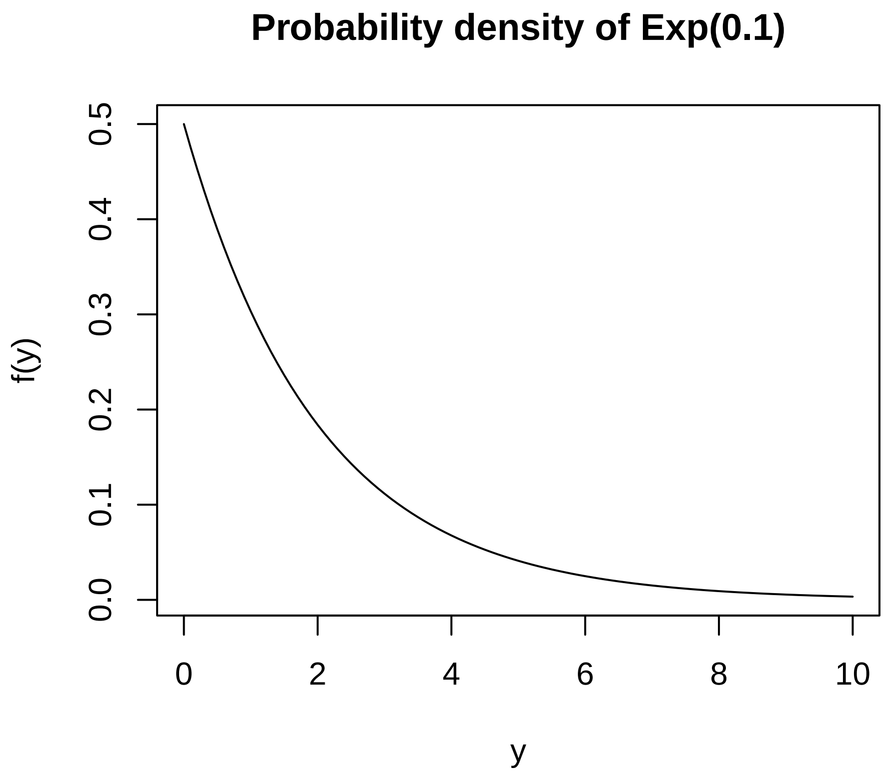

Figure 2.11: Exponential probability distribution

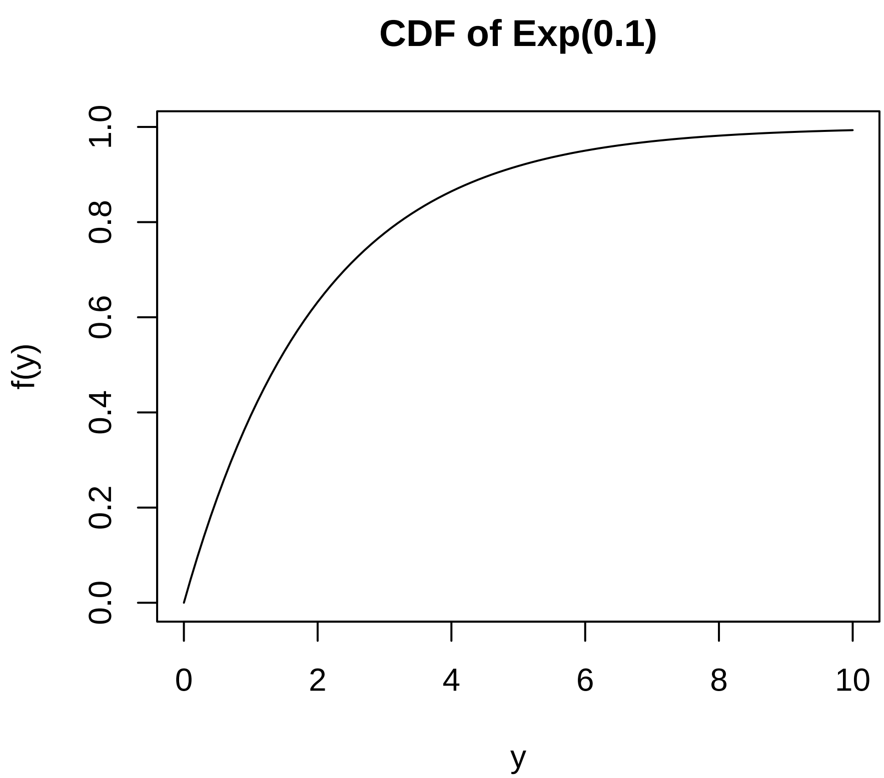

Figure 2.12: Exponential cumulative probability distribution

```
## [1] 1.990456
```

```
hist(y, main="Histogram of draws from Exp(0.5)")
```

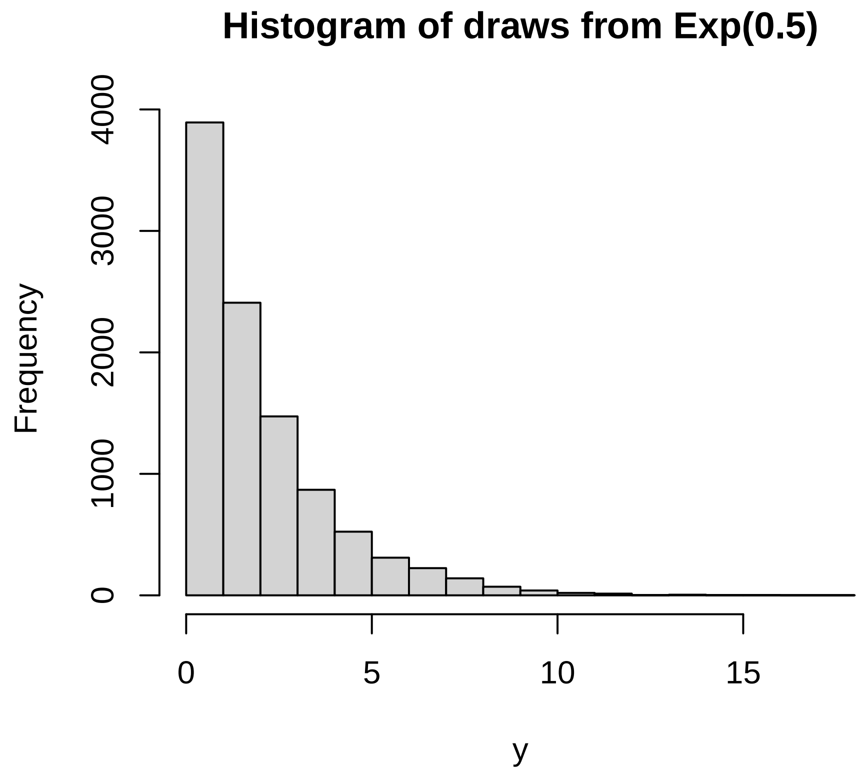

Figure 2.13: 10000 draws from an Exponential distribution

2.5 Sampling from a finite collection

The simulations discussed so far all draw from **infinite** populations: we can set the sample size n to any value. However it often occurs that we have only a finite population - e.g. when drawing a random sample of students from a class, or households from a suburb for interview.

Consider the case where we have a street of $N = 10$ households, numbered 1-10. We want to select 5 of these with equal probability. There are a number of ways we could do this.

Sampling without replacement

We draw a random number from 1-10

Chapter 3

The Survey Process

Steps in survey sampling

3.1 Populations

3.2 Frames

Chapter 4

Sample Designs

Theory, and how to do this in R

- SRSWOR, SRSWR
- Stratified designs
- Cluster designs
- PPSWR
- Complex designs

Introduction to the design effect

Chapter 5

Inference with R

Data display and inference

Analytic and jackknife methods

- SRSWOR, SRSWR
- Stratified designs
- Cluster designs
- PPSWR
- Complex designs

Chapter 6

Missing data

Origin of missing data

Calibration for unit non-response

- Poststratification

Imputation for item non-response

References

Appendix A

Computing the Required Sample Size

1. Identify your **design estimate** – this is the estimate that will underlie a headline you could imagine being written about your survey.

2. Identify the desired MOE, m, for this design estimate, noting whether you want this MOE to apply to the overall estimate, or for that estimate in each of, say, H subgroups/strata.

3. Compute the sample size required, n_1, assuming that you're going to select a Simple Random Sample and ignoring the fpc.

   For estimation of a mean $\bar{Y}$ of some characteristic y:

   $$n_1 = \left(\frac{Z}{MOE}\right)^2 s^2$$

   where Z is appropriate for the desired level of confidence and s is the estimated population standard deviation of the characteristic y.

   For estimation of a proportion P:

   $$n_1 = \left(\frac{Z}{MOE}\right)^2 p(1-p)$$

   where p is the estimated proportion, or 0.5 if really unknown.

4. If the sample size is a substantial proportion of the population or subgroup size N, then apply the fpc adjustment:

   $$n_2 = \frac{n_1}{1 + \frac{n_1}{N}}$$

5. If you have H subgroups and are using equal allocation then multiply by H

   $$n_3 = Hn_2$$

   or alternatively sum over the n_2 values in each of the strata you've defined.

6. Apply the design effect (Deff) appropriate for the actual design you'll be using.

   $$n_4 = \text{Deff} \times n_3$$

7. Adjust for anticipated non-response: if response rate ϕ is expected then

$$n_5 = \frac{n_4}{\phi}$$

   (If non-response is likely to vary between strata, then apply this correction to n_2, within each stratum.)

Bibliography

Cochran, W. G. (1977). *Sampling Techniques*. Wiley, New York, third edition.

Kish, L. (1965). *Survey Sampling*. Wiley, New York.

Little, R. J. A. and Rubin, D. B. (2002). *Statistical Analysis with Missing Data*. Wiley, Hoboken, second edition.

Lohr, S. L. (1999). *Sampling: Design and Analysis*. Duxbury Press, Pacific Grove, CA, USA.

Lumley, T. (2004). Analysis of complex survey samples. *Journal of Statistical Software*, 9(1):1–19. R package verson 2.2.

Lumley, T. S. (2010). *Complex Surveys: A Guide to Analysis Using R (Wiley Series in Survey Methodology)*. Wiley.

Särndal, C.-E., Swensson, B., and Wretmann, J. (1992). *Model Assisted Survey Sampling*. Springer-Verlag, New York.

Scheaffer, R. L., III, W. M., and Ott, R. L. (2006). *Elementary Survey Sampling*. Duxbury, Belmont, sixth edition.

Zealand, S. N. (1995). *A Guide to Good Survey Design*. Statistics New Zealand, Wellington, second edition.